

Континуирана интеграција и испорука сервиса без серверских инстанци на AWS платформи

Душан Ђорђевић

Завршни рад

2025

Мотивација и циљ

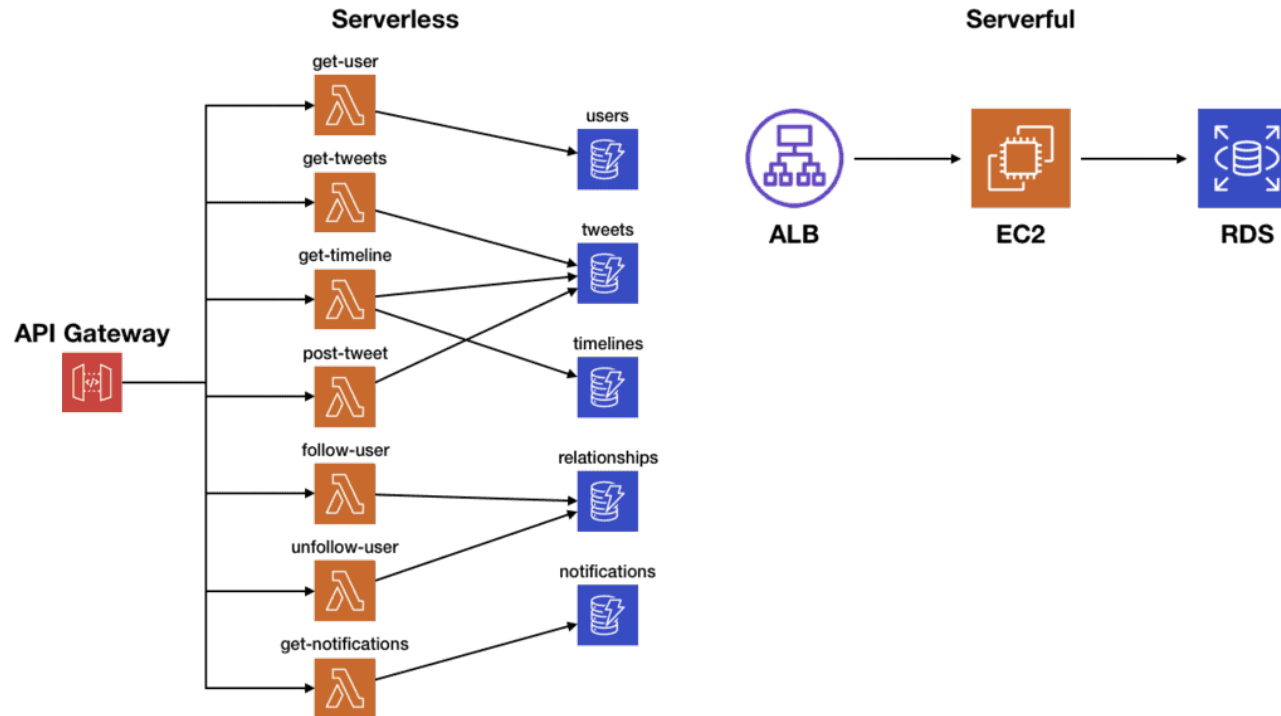
- аутоматизација процеса развоја и постављања *streaming* апликације
- елиминација ручних и грешкама подложних корака
- омогућавање честих и поузданих испорука
- рано откривање грешака кроз аутоматске провере
- обезбеђивање конзистентности између окружења
- дефинисање инфраструктуре као кода

Теоријске основе

Безсерверска (енгл. *serverless*) архитектура

- нема трајно активне инфраструктуре
- плаћање само за коришћене ресурсе
- аутоматско скалирање
- догађајно оријентисан модел

Безсерверска (енгл. *serverless*) архитектура



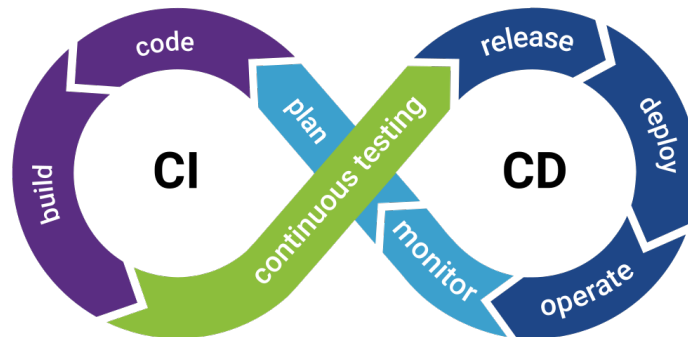
Континуирана интеграција и испорука (енгл. *CI/CD*)

Континуирана интеграција (CI)

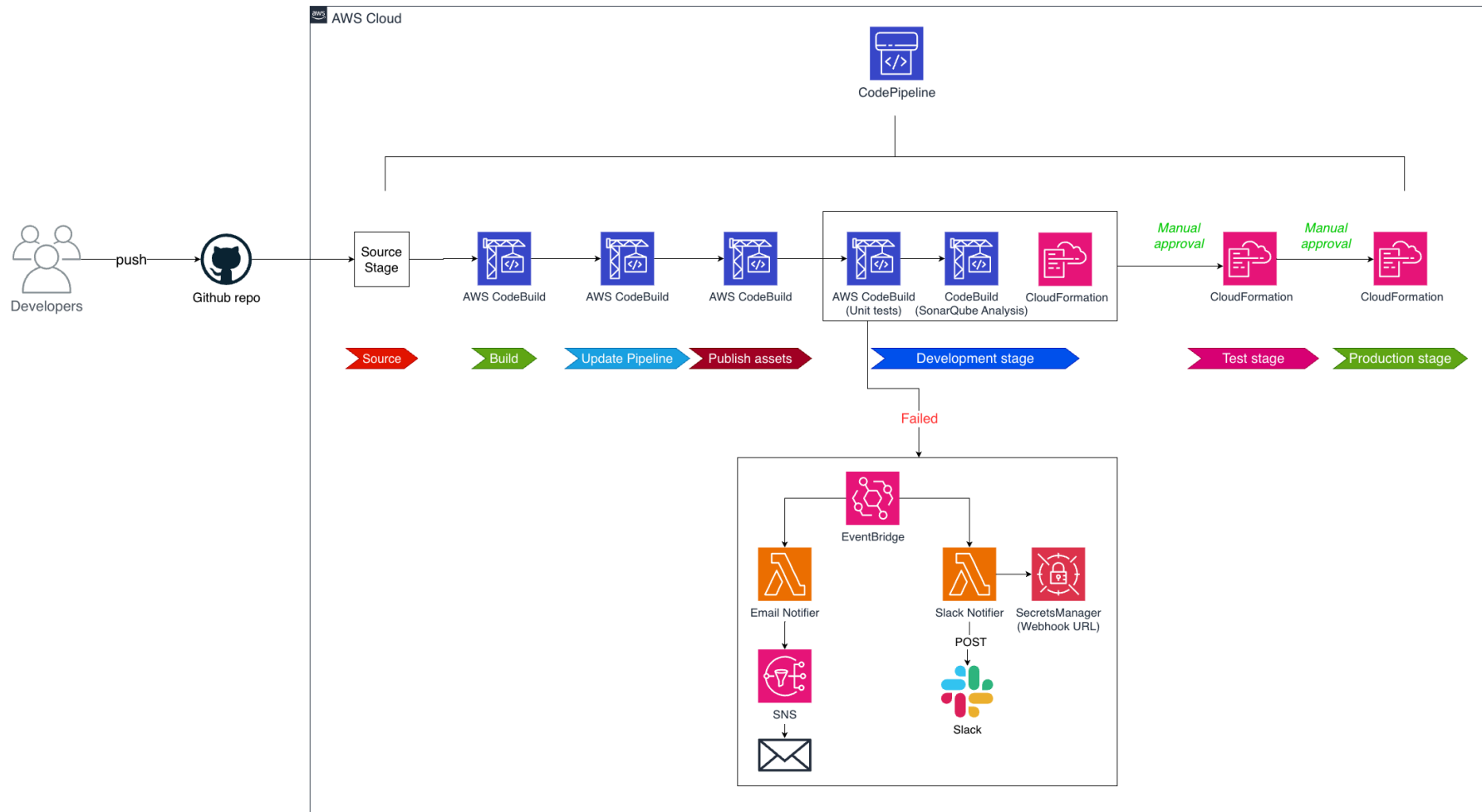
- често интегрисање измена
- аутоматско грађење
- јединични тестови
- статичка анализа кода

Континуирана испорука (CD)

- аутоматско постављање
- више окружења (*DEV, TEST, PROD*)
- ручна одобрења
- брз опоравак (*rollback*)



Архитектура



Имплементација система

Технологије

- *AWS CDK (TypeScript)*
- *AWS CLI*
- *GitHub*
- *SonarQube*
- *pytest*
- *Slack*



aws
**Cloud
Development
Kit** (TypeScript)



 **SonarQube**



Структура пројекта

```
infrastructure/
├── bin/
│   └── infrastructure.ts
├── lib/
│   ├── cird-stack.ts
│   ├── pipeline.ts
│   └── stacks/
├── lambda/
│   └── event-invoked/
├── test/unit/
├── cdk.json
└── package.json
```

- **bin/** - улазна тачка *CDK* апликације
- **lib/** - дефиниције *stack*-ова
- **lambda/** - *Lambda* функције
- **test/unit/** - јединични тестови
- **cdk.json** - *CDK* конфигурација
- **package.json** - зависности

Креирање *CodePipeline*-а

```
const pipeline = new CodePipeline(this, 'Pipeline', {
  pipelineName: 'Pipeline',
  synth: new ShellStep('Synth', {
    input: CodePipelineSource.gitHub(
      'duledjordjevic/streamio',
      'main'
    ),
    commands: [
      'cd infrastructure',
      'npm ci',
      'npx cdk synth',
    ],
    primaryOutputDirectory: 'infrastructure/cdk.out'
  })
});
```

SonarQube интеграција

```
const sonarToken = secretsmanager.Secret.fromSecretNameV2(  
    this, 'SonarToken', 'sonarqube-token'  
);  
  
const sonarQubeStep = new CodeBuildStep('SonarQube Analysis', {  
    commands: [  
        'export SONAR_SCANNER_VERSION=7.2.0.5079',  
        // ... преузимање SonarQube Scanner-a  
        'sonar-scanner -Dsonar.projectKey=... -Dsonar.token=$SONAR_TOKEN'  
    ],  
    buildEnvironment: {  
        environmentVariables: {  
            SONAR_TOKEN: {  
                type: BuildEnvironmentVariableType.SECRETS_MANAGER,  
                value: sonarToken.secretArn  
            }  
        }  
    }  
});
```

Дефинисање окружења

```
const devStage = pipeline.addStage(  
  new PipelineStage(this, 'PipelineDevStage', { stageName: 'dev' })  
);  
devStage.addPre(sonarQubeStep);  
devStage.addPre(unitTestsStep);  
  
const qaStage = pipeline.addStage(  
  new PipelineStage(this, 'PipelineQAStage', { stageName: 'qa' })  
);  
qaStage.addPre(new ManualApprovalStep('ApproveQA'));  
  
const prodStage = pipeline.addStage(  
  new PipelineStage(this, 'PipelineProdStage', { stageName: 'prod' })  
);  
prodStage.addPre(new ManualApprovalStep('ApproveProd'));
```

Систем обавештавања

EventBridge правила

- праћење *CodePipeline* догађаја (*STARTED*, *SUCCEEDED*, *FAILED*)
- праћење *CodeBuild* неуспеха
- активирање *Lambda* функција

Канали обавештавања

- *Slack* - све промене стања *pipeline*-а, боја према статусу
- *Email* - само критични неуспеси преко *SNS* теме

Закључак

Предности

- аутоматизација процеса
- доследност и поновљивост
- брзо обавештавање о грешкама
- лако праћење статуса

Мане

- зависност од AWS услуга
- време извршавања *pipeline*-а

Могућа унапређења

- аутоматизовани интеграциони тестови у тест окружењу
- додатне метрике и алармирање (*CloudWatch, X-Ray...*)
- *canary deployment* стратегија
- *blue-green deployment*
- логовање и праћење (*ELK Stack, CloudWatch Logs...*)

Хвала на пажњи

Питања?